PDF Page Extractor: A-to-Z Coding Guide

PDF Page Extractor: A-to-Z Coding Guide

This guide explains how to code the full-stack PDF Page Extractor from start to end. Use it as your own learning roadmap and interview explanation.

1. What We Are Building

We are building a web app where a user can:

- Upload a PDF file.
- See all pages as visual previews.
- Select specific pages.
- Rearrange selected pages.
- Generate a new PDF.
- Download the extracted PDF.

The project has two parts:

- Backend: Node.js, Express, TypeScript, Multer, PDF-lib.
- Auth: JWT tokens, bcrypt password hashing, OTP email verification.
- Frontend: React, Vite, TypeScript, PDF.js, Axios, Framer Motion.

2. Final Folder Structure

pdf..extractor/
backend/
index.ts
package.json
tsconfig.json
.env.example
src/
app.ts
config/
multer.ts
constants/
config.ts
messages.ts
routes.ts
statusCodes.ts
controllers/
authController.ts
pdfController.ts
dtos/
authDtos.ts
pdfDtos.ts
mappers/
pdfMapper.ts
userMapper.ts
middleware/
authenticate.ts
repositories/

jsonFileRepository.ts
pdfRepository.ts
userRepository.ts
routes/
authRoutes.ts
pdfRoutes.ts
services/
authService.ts
emailService.ts
pdfService.ts
types/
models.ts
utils/
cleanup.ts
test/
pdfService.test.ts
uploads/
outputs/

frontend/
package.json
.env.example
index.html
src/
main.tsx
constants/
api.ts
messages.ts

App.tsx

App.css

index.css

docs/
pdf-extractor-a-to-z-guide.md
pdf-extractor-a-to-z-guide.pdf
screenshots/
upload-state.png

README.md

3. Backend Architecture

The backend uses a clean layered structure:

- index.ts: starts the server.
- app.ts: uses AppFactory and ErrorHandlerMiddleware classes to configure Express middleware, routes, static files, cleanup job, and error handling.
- routes/pdfRoutes.ts: uses PdfRoutes class to map API endpoints to controller methods.
- routes/authRoutes.ts: uses AuthRoutes class to map authentication endpoints.
- controllers/pdfController.ts: class-based PDF HTTP orchestration.
- controllers/authController.ts: class-based auth HTTP orchestration.
- dtos/authDtos.ts: validates and normalizes auth request bodies.
- dtos/pdfDtos.ts: validates PDF extraction request bodies and maps page numbers to PDF-lib indexes.

- mappers/userMapper.ts: converts stored user records into public user responses.
- mappers/pdfMapper.ts: converts PDF upload/extract data into storage records and response DTOs.
- services/pdfService.ts: contains the actual PDF business logic.
- services/authService.ts: contains password hashing, OTP, and JWT logic.
- services/emailService.ts: sends OTP email or logs dev OTP when SMTP is not configured.
- constants: stores shared routes, status codes, messages, limits, and storage names.
- middleware/authenticate.ts: uses AuthenticationMiddleware class to validate JWT tokens for protected routes.
- repositories: stores user records and PDF ownership records in JSON files.
- config/multer.ts: uses MulterConfig class to configure file upload validation and storage.
- utils/cleanup.ts: uses CleanupJob class to delete old uploaded/generated files.
- test/pdfService.test.ts: tests the PDF extraction logic.

Simple request flow:

Frontend request

-> Express route
-> Auth middleware for protected routes
-> Controller class method
-> DTO validator
-> PDF service
-> Mapper
-> File system / PDF-lib
-> Controller response
-> Frontend UI update

SOLID responsibility split:

- Single Responsibility: routes route, controllers orchestrate HTTP, DTO validators validate input, mappers transform data, services run business rules, repositories persist data.
- Open/Closed: adding a new request shape means adding or extending a DTO validator without rewriting service internals.
- Liskov Substitution: validators, mappers, repositories, and services are consumed by narrow contracts, so equivalent implementations can replace them in tests.
- Interface Segregation: each layer exposes only the methods its caller needs.
- Dependency Inversion: controllers and setup classes depend on service/validator/mapper/repository collaborators through constructor parameters or small class fields, which keeps logic easy to test and replace.

4. Backend Step-by-Step

Step 1: Initialize Backend

mkdir backend
cd backend
npm init -y

Install runtime dependencies:

npm install express cors multer pdf-lib dotenv node-cron

Install TypeScript development dependencies:

npm install -D typescript tsx @types/node @types/express @types/cors @types/multer @types/node-cron

Step 2: Configure TypeScript

Create backend/tsconfig.json.

Important settings:

- strict: true catches mistakes early.
- outDir: dist puts compiled JavaScript in dist.
- module: Node16 works well with Node TypeScript projects.

Step 2.5: Create Constants

Create a constants folder before writing controllers and services:

src/constants/routes.ts
src/constants/messages.ts
src/constants/statusCodes.ts
src/constants/config.ts

Use constants for:

- API base routes.
- Auth routes.
- PDF routes.
- Response status codes.
- Error and success messages.
- OTP limits.
- File size limits.
- Upload/output folder names.

Learning point:

- Constants avoid repeated strings across the codebase.
- If a route or message changes, you update one file instead of hunting through controllers and services.
- This improves code quality and makes your project easier to explain.

Step 2.6: Create DTO Validators And Mappers

Create DTO and mapper folders before wiring controllers:

src/dtos/authDtos.ts
src/dtos/pdfDtos.ts
src/mappers/userMapper.ts
src/mappers/pdfMapper.ts

Use DTO validators for:

- Request body shape checks.
- Trimming and normalizing strings.
- Converting user-facing page numbers to zero-based PDF-lib page indexes.
- Rejecting invalid data before it reaches business logic.

Use mappers for:

- Converting UserRecord into PublicUser.
- Converting uploaded PDF metadata into PdfRecord.
- Converting stored records into API response DTOs.

Learning point:

- Validation and mapping are not the same as business logic. Keeping them separate makes the service cleaner and keeps every class focused on one job.

Step 3: Create Entry Point

index.ts loads environment variables and starts the server.

Learning point:

- app.listen(PORT) is the line that makes your backend available in the browser/API client.

Step 4: Create Express App

src/app.ts does these jobs:

- Creates Express app.
- Enables CORS.
- Enables JSON body parsing.
- Serves generated PDFs from /outputs.
- Mounts PDF routes at /api/pdfs.
- Starts cleanup job.
- Handles errors globally.

Learning point:

- app.use('/api/pdfs', pdfRoutes) means every route inside pdfRoutes starts with /api/pdfs.

Step 5: Configure Multer Upload

src/config/multer.ts:

- Stores uploaded files in uploads.
- Gives every file a unique name using crypto.randomUUID.
- Allows only PDF MIME type and .pdf extension.
- Limits file size to 50 MB.

Learning point:

- Multer handles multipart/form-data, which normal express.json() cannot handle.

Step 6: Create Routes

src/routes/pdfRoutes.ts:

POST /api/pdfs/upload
GET /api/pdfs/:id
POST /api/pdfs/:id/extract

Learning point:

- Routes should stay small. They only connect URLs to controller functions.

Step 7: Create Controller Logic

src/controllers/pdfController.ts is a class with three public methods:

- uploadPdf: validates uploaded file, reads PDF page count, returns metadata.
- getPdf: sends uploaded PDF back for preview.
- extractPdfPages: validates selected pages through PdfDtoValidator, calls service, saves output PDF, returns a mapped download DTO.

Learning point:

- Controller understands HTTP.
- DTO validator understands request body rules.
- Mapper understands response and storage transformation.
- Service understands business logic.
- Keep these separate so your code is easier to explain.

Step 8: Create PDF Service

src/services/pdfService.ts uses pdf-lib.

Core logic:

Read original PDF bytes.
Load PDF using PDFDocument.load.
Create a new PDF using PDFDocument.create.
Copy selected pages in requested order.
Add copied pages to new PDF.
Save new PDF bytes.
Return Buffer.

Learning point:

- The frontend sends page numbers like [3, 1].
- pdf-lib uses zero-based indexes, so backend converts that to [2, 0].

Step 9: Cleanup Strategy

src/utils/cleanup.ts runs every hour.

It deletes files older than the configured limit from:

- uploads

- outputs

Learning point:

- Uploaded files should not stay forever on the server.

Step 10: Backend Tests

test/pdfService.test.ts creates a sample PDF in memory and tests:

- Selected pages create a new PDF.
- Invalid page ranges fail.

test/dtoValidation.test.ts tests:

- Auth DTO normalization.
- OTP format validation.
- PDF page number mapping from one-based to zero-based indexes.
- Rejection of invalid pages payloads.

Run:

npm test

5. Frontend Architecture

The frontend is a Vite React TypeScript app.

Main files:

- src/main.tsx: renders React app.
- src/constants/api.ts: stores API base URL, endpoint paths, token key, and auth header names.
- src/constants/messages.ts: stores reusable UI fallback messages.
- src/App.tsx: contains upload, preview, selection, reorder, extract, and download logic.
- src/App.css: component-level UI styles.
- src/index.css: global theme styles.

6. Frontend Step-by-Step

Step 1: Initialize Frontend

npm create vite@latest frontend -- --template react-ts
cd frontend
npm install

Install dependencies:

npm install axios pdfjs-dist framer-motion lucide-react

Step 2: Configure API URL

Create frontend/.env.example:

VITE_API_URL=http://localhost:5000

Learning point:

- Vite exposes frontend environment variables only when they start with VITE_.

Step 3: Build Upload UI

The upload UI accepts:

- Drag and drop.
- File picker.
- Only PDF files.

When a file is selected:

Create FormData.

Append file as "pdf".
POST to /api/pdfs/upload.
Store returned metadata in React state.

Step 4: Render PDF Page Previews

The frontend uses pdfjs-dist.

For each page:

Load the uploaded PDF URL.
Get page by number.
Create viewport.
Render page into canvas.

Learning point:

- pdf-lib edits PDFs on the backend.
- pdfjs-dist displays PDFs on the frontend.

Step 5: Select Pages

React state:

const [selectedPages, setSelectedPages] = useState<number[]>([])

If user clicks page 4:

- If page 4 is not selected, add it.
- If page 4 is already selected, remove it.

Step 6: Rearrange Pages

Selected pages are stored in order.

Example:

Selected: [3, 1, 5]
Output PDF order: page 3, then page 1, then page 5

Move up/down buttons swap positions inside the array.

Step 7: Extract Pages

When user clicks Extract:

POST /api/pdfs/:id/extract
Body: { pages: selectedPages }

Backend creates new PDF.
Frontend receives downloadUrl.
Show Download button.

Step 8: Download Flow

The backend returns:

{
"fileName": "generated-id-extracted.pdf",
"pageCount": 2,
"downloadUrl": "/outputs/generated-id-extracted.pdf"
}

Frontend creates a full URL:

http://localhost:5000/outputs/generated-id-extracted.pdf

Then the user downloads the file.

7. Error Handling

Backend handles:

- Missing PDF.
- Wrong file type.
- Invalid page array.
- Page outside PDF range.
- Unknown server errors.

Frontend handles:

- Invalid selected file.
- Upload failure.
- Backend not reachable.

- Extraction failure.
- Empty selected pages.

8. How To Run The Project

Open terminal 1:

```
cd backend
npm install
copy .env.example .env
npm run dev
```

Backend URL:

http://localhost:5000

Open terminal 2:

```
cd frontend
npm install
copy .env.example .env
npm run dev
```

Frontend URL:

http://localhost:5173

If PowerShell blocks npm, use:

```
npm.cmd run dev
```

9. How To Test

Backend:

```
cd backend
npm test
```

Backend TypeScript build:

```
cd backend
npm run build
```

Frontend build:

```
cd frontend
npm run build
```

Frontend lint:

```
cd frontend
npm run lint
```

10. How To Explain In Interview

Short explanation:

I built a full-stack PDF page extractor. The frontend uploads PDFs, renders page previews with PDF.js, lets users select and reorder pages, then sends the selected page numbers to the backend. The backend is built with Express and TypeScript. It validates uploads using Multer, processes PDFs using PDF-lib, creates a new PDF with selected pages, stores it temporarily, and returns a download URL. I also added cleanup for old files, centralized error handling, and backend tests for PDF extraction.

Architecture explanation:

I separated backend code into routes, controllers, DTO validators, mappers, services, repositories, config, and utilities. Routes define endpoints. Controller class methods handle HTTP orchestration. DTO validators validate and normalize request bodies. Mappers transform records into response DTOs. Services contain business logic. This makes PDF extraction, validation, and mapping testable without needing Express.

PDF extraction explanation:

The frontend sends one-based page numbers because that is natural for users. The backend converts them to zero-based indexes because PDF-lib expects zero-based page positions. Then PDF-lib copies pages in the same order received, so rearranging pages is naturally supported.

11. Build Order If Coding From Scratch

Follow this exact order:

1. Create backend project.
2. Install backend dependencies.
3. Add TypeScript config.
4. Create index.ts.
5. Create app.ts.
6. Create constants for routes, messages, status codes, and config limits.
7. Create repositories for users and PDFs.
8. Create auth service with bcrypt, OTP, and JWT.
9. Create email service with Nodemailer.
10. Create auth controller and auth routes.
11. Create auth middleware.
12. Create upload config with Multer.
13. Create protected PDF routes.
14. Create PDF controller functions.
15. Create PDF service.
16. Add cleanup job.
17. Add backend tests.
18. Create frontend project.
19. Install frontend dependencies.
20. Create frontend constants for API routes and messages.
21. Create auth UI.
22. Store JWT token.
23. Create upload UI.
24. Connect upload API with JWT.
25. Render page previews with PDF.js and JWT headers.

26. Add page selection.
27. Add page reorder.
28. Connect extraction API.
29. Add download flow.
30. Add responsive CSS.
31. Add README and screenshots.
32. Run tests and builds.
33. Push to GitHub.
34. Deploy if needed.

12. Most Important Concepts To Learn

- REST API basics.
- File upload using Multer.
- Difference between controller and service.
- PDF editing with PDF-lib.
- PDF rendering with PDF.js.
- React state for selected pages.
- TypeScript types for safer backend/frontend code.
- Error handling.
- Temporary file cleanup.
- README and submission quality.

13. Common Mistakes To Avoid

- Sending page numbers directly to PDF-lib without converting to zero-based indexes.
- Forgetting to validate file type.
- Forgetting to create uploads and outputs folders.
- Mixing PDF rendering and PDF editing libraries.
- Keeping business logic inside controllers.
- Not returning clear errors to the frontend.
- Not documenting run commands.
- Not testing the PDF service.

14. Final Checklist

- Upload PDF works.
- Non-PDF upload is rejected.
- All pages preview correctly.
- Page selection works.
- Page reorder works.
- Extraction creates correct PDF.
- Download link works.
- Signup works.
- OTP verification works.
- Resend OTP handles cooldown.
- Login works only after email verification.
- JWT protects PDF routes.
- Uploaded PDFs belong to the logged-in user.
- App is responsive.
- Backend tests pass.
- Frontend build passes.
- README has run commands.
- Screenshots are included.

15. Authentication Add-On: Signup, OTP, Login, JWT

The authentication feature adds a secure user layer on top of the PDF extractor.

Backend Files Added

backend/src/controllers/authController.ts
backend/src/controllers/pdfController.ts
backend/src/dtos/authDtos.ts
backend/src/dtos/pdfDtos.ts
backend/src/mappers/pdfMapper.ts
backend/src/mappers/userMapper.ts
backend/src/middleware/authenticate.ts
backend/src/constants/config.ts
backend/src/constants/messages.ts
backend/src/constants/routes.ts
backend/src/constants/statusCodes.ts
backend/src/repositories/jsonFileRepository.ts
backend/src/repositories/userRepository.ts
backend/src/repositories/pdfRepository.ts
backend/src/routes/authRoutes.ts
backend/src/services/authService.ts
backend/src/services/emailService.ts
backend/src/types/models.ts

Auth API Routes

POST /api/auth/signup
POST /api/auth/verify-otp
POST /api/auth/resend-otp
POST /api/auth/login
GET /api/auth/me

Signup Flow

User enters name, email, password.
Backend validates name, email, and password.
Backend checks duplicate verified email.
Backend hashes password using bcrypt.
Backend generates a 6 digit OTP.
Backend hashes OTP before storing.
Backend stores OTP expiry, attempts, and last sent time.
Backend sends OTP by email using Nodemailer.
If SMTP is not configured in development, OTP is returned as devOtp.

OTP Verification Flow

User enters email and OTP.
Backend checks account exists.
Backend checks account is not already verified.

Backend checks OTP exists.
Backend checks OTP is not expired.
Backend checks max attempts.
Backend compares entered OTP with hashed OTP using bcrypt.
If valid, account becomes verified.
Backend clears OTP fields.
Backend returns JWT token.

Login Flow

User enters email and password.
Backend finds user by email.
Backend compares password with passwordHash.
Backend blocks login if email is not verified.
Backend signs JWT token.
Frontend stores token in localStorage.

JWT Flow

Frontend sends Authorization: Bearer <token>.
authenticate middleware reads the token.
Backend verifies token with JWT_SECRET.
Backend attaches userId and email to req.user.
Protected PDF controllers use req.user.userId.

PDF Ownership

When a user uploads a PDF, the backend saves a PDF record:

id
userId
originalName
size
pageCount
path
createdAt

When the user previews or extracts pages, the backend checks:

Does this PDF exist?
Does this PDF belong to the logged-in user?
Does the file still exist on disk?

Auth Edge Cases Handled

- Invalid email format.
- Short password.
- Duplicate verified email.
- Login before OTP verification.
- Invalid password.

- Account not found.
- OTP expired.
- Too many OTP attempts.
- OTP resend cooldown.
- Already verified account.
- Missing JWT.
- Invalid or expired JWT.
- User trying to access another user's PDF.

Environment Variables For Auth

```
JWT_SECRET=replace-with-a-long-random-secret
JWT_EXPIRES_IN=7d
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
SMTP_FROM=
```

For local learning, SMTP can stay empty. The OTP appears as devOtp in the response and is also printed in the backend console. For production, configure SMTP and keep OTP hidden from responses.